Clasificación del radical de kanji dibujados a mano

Una CNN propia frente a una ResNet50 preentrenada en ImageNet

Amir Canto Diego Roman Jarem Piero Castillo

16 de octubre de 2026

Computación Gráfica · Práctica Calificada 2
Universidad Nacional de Ingeniería

Contenido

1. Problema y datos

2. Preprocesado y aumento de datos

3. Modelos y entrenamiento

4. Resultados

5. Conclusiones

Problema y datos

El problema

Dado un kanji **dibujado a mano**, predecir cuál de **5 radicales** contiene.



- Hay **43 kanji** y solo 5 clases: la red no tiene que reconocer el kanji, sino el radical que comparte con otros (池, 海 y 湖 son todos `water`).
- El radical se elige por **apariencia**: 畑 va en `fire` porque lleva 火 a la izquierda, aunque el diccionario Kangxi lo clasifique bajo 田.

Clases y kanji

Clase	Radical	Nº kanji	Kanji
earth	土	9	地 坂 均 坊 城 型 域 場 坪
fire	火	8	灯 炉 炊 炒 畑 炎 炭 灰
metal	金	8	針 釣 鈴 鉄 鉛 銀 銅 銭
tree	木	9	机 村 材 林 板 松 校 桜 杉
water	氵	9	池 河 油 泳 波 海 洋 流 湖

El radical puede ir a la izquierda (地, 針), abajo (型, 炭) o repetido (林, 炎).

Recolección: una app de dibujo propia

- App **Flask** (`make draw`): muestra el kanji que toca, con el orden de trazos de referencia.
- Lienzo de **300×300** y pincel de **10 px**.
- **4 escritores**: cada uno dibuja cada kanji 9 veces (`water`, `tree`, `earth`) o 10 (`fire`, `metal`).
- Cada escritor empieza por una clase distinta.

Cómo se guarda

PNG RGBA: el trazo solo vive en el **canal alfa**.

```
drawings/<clase>/<kanji>/
  <escritor>-<NN>.png
```

p. ej. `drawings/fire/畑/diego-03.png`

Etiqueta = carpeta

La carpeta de primer nivel es la clase; la intermedia, el kanji; el nombre del archivo da el escritor y la repetición.

El mismo kanji, cuatro escritores



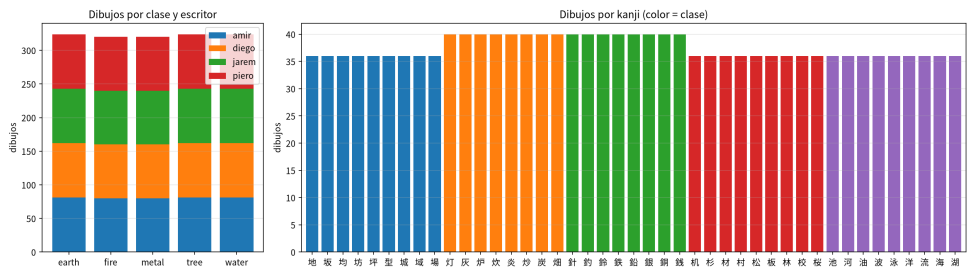
Un kanji por radical (3.ª repetición de cada escritor).

Cambian de un escritor a otro:

- tamaño y posición en el lienzo,
- proporciones entre radical y resto,
- inclinación y forma de cada trazo.

El preprocesado elimina lo primero; lo demás lo tiene que aprender la red.

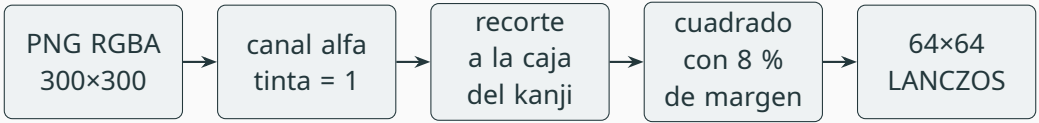
Dataset



1612 dibujos **403** por escritor **320–324** por radical **36–40** por kanji

Preprocesado y aumento de datos

Preprocesado



- El recorte y el centrado quitan el efecto de **dónde y a qué tamaño** se dibujó, pero el padding cuadrado **no deforma** el kanji: se conserva dónde está el radical (izquierda, abajo…).
- A 64×64 el pincel de 10 px queda en unos **2 px**.
- Salida: tensor $1 \times 64 \times 64$ en $[0, 1]$. Los 1612 dibujos se cargan en memoria (unos 26 MB).
- Manifiesto `data/build/drawings_index.csv`: ruta, etiqueta, kanji, escritor y split de cada dibujo.

División train / val / test

70 / 15 / 15, **estratificado por kanji**: cada kanji (y cada radical) aparece en los tres conjuntos en la misma proporción.

Conjunto	Dibujos	Uso
train	1128	ajuste de pesos
val	242	LR y mejor época
test	242	solo métricas finales

Semilla 42. Cada corrida guarda una **huella** (sha1 de las rutas): si cambian los dibujos, no se mezclan corridas.

Aumento de datos: 4 transformaciones

Solo en train, sobre la imagen ya preprocesada (64×64). Cada dibujo genera una variante de cada tipo:

Transformación	Qué hace	Parámetro
ruido gaussiano	suma $\mathcal{N}(0, \sigma)$ a cada píxel	$\sigma \in [0.05, 0.15]$
sal y pimienta + media	p de los píxeles a 0 o 1; media 3×3	$p \in [0.02, 0.06]$
sal y pimienta + mediana	el mismo ruido; mediana 3×3	$p \in [0.02, 0.06]$
rotación	giro alrededor del centro	$\pm 3°$ a $\pm 15°$

- Sobre el PNG de 300×300, el ruido repartido por el lienzo arruinaría el recorte a la caja del kanji.
- Filtros de 3×3: con trazos de unos 2 px, uno de 5×5 los borraría.
- Sin volteos: cambiarían el lado del radical (氵 siempre va a la izquierda).

Ejemplos del aumento



Un dibujo por radical: original → 64×64 → sus 4 variantes, con sus parámetros.

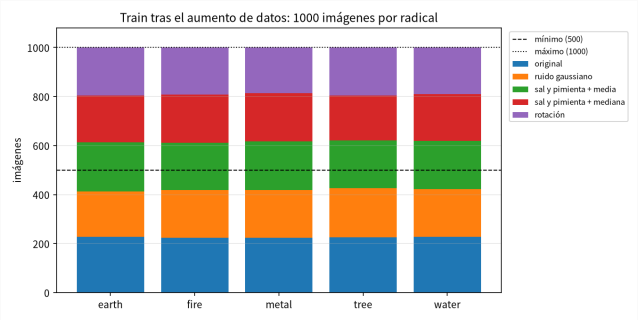
- La **mediana** elimina casi todo el ruido impulsivo sin difuminar el trazo.
- La **media** lo difumina, pero no lo quita.
- La rotación no recorta el kanji gracias al margen del 8 %.

Cuántas imágenes: entre 500 y 1000 por radical

Objetivo común para todas las clases, con n_c = originales de train del radical c:

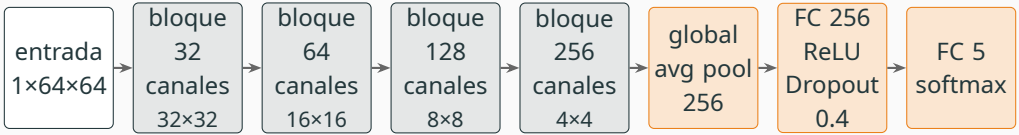
$$T = \text{mín}\big(\text{máx}(5 \cdot \text{máx}_c\, n_c,\ 500),\ 1000\big)$$

- $n_c \approx 227$:
  $5 \cdot 227 = 1135 > 1000$, así que $T = \mathbf{1000}$.
- Train: **5000 imágenes**, clases iguales.
- Cada original queda con 3 o 4 variantes.
- Registro: `train_augmented.csv`.



Modelos y entrenamiento

Nuestra CNN: `KanjiRadicalCNN`



Cada bloque

[Conv $3\times3 \rightarrow$ BatchNorm $\rightarrow$ ReLU] $\times 2 \rightarrow$ MaxPool 2×2

- **1.24 M** parámetros (4.7 MB).
- El *global average pooling* evita una capa densa enorme sobre el mapa de 4×4.

La referencia: ResNet50 preentrenada en ImageNet

- `torchvision` con pesos `IMAGENET1K_V2`; se ajustan **todas** las capas.
- **1 canal**: los pesos RGB de `conv1` se suman. Da la misma respuesta que la convolución original sobre la imagen gris replicada en los 3 canales.
- Normalización gris equivalente a ImageNet: media 0.449, desviación 0.226.
- `fc`: 2048 $\rightarrow$ 5.

¿Por qué compararla?

Con un dataset pequeño, el *transfer learning* desde ImageNet es lo que suele recomendarse. Si la CNN propia la iguala, una red 19 veces más pequeña basta.

parámetros	23.5 M
tamaño	89.9 MB

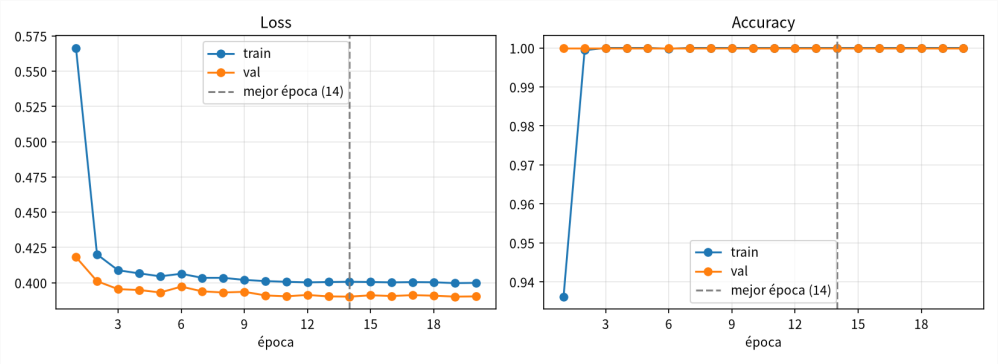
Entrenamiento

optimizador	Adam
learning rate inicial	10^{-3} (CNN) 10^{-4} (ResNet50, ya preentrenada)
pérdida	cross-entropy con *label smoothing* 0.1
scheduler	`ReduceLROnPlateau` sobre `val_loss` (factor 0.5, paciencia 3)
épocas y batch	20 épocas, batches de 32 (5000 imágenes por época)
modelo final	pesos de la época con menor `val_loss`

Las dos redes usan **el mismo split, el mismo train aumentado y el mismo test**: la huella del dataset se comprueba antes de comparar.

Resultados

CNN: curvas de entrenamiento

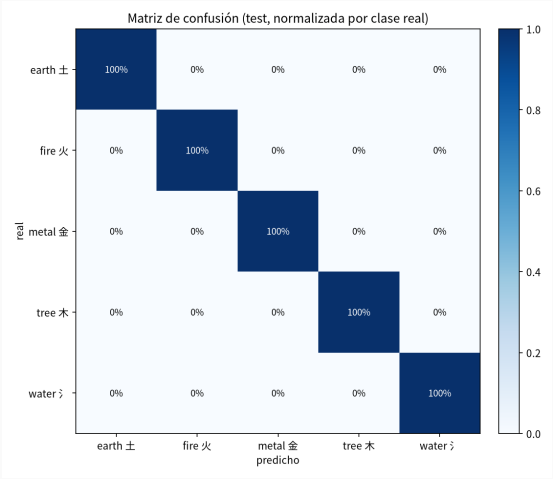


- Val acc = 1 desde la primera época; la pérdida sigue bajando hasta la **época 14** (menor `val_loss`).
- La pérdida no baja de ≈ 0.39 por el *label smoothing*. Unos 35 s por época en CPU.

CNN: test (242 dibujos)

Métrica	CNN
accuracy	1.000
precision (macro)	1.000
recall (macro)	1.000
F1 (macro)	1.000
loss	0.390
inferencia	2.5 ms/img

Acierta los 242 dibujos de test, de los 43 kanji y los 4 escritores.



Cómo se compara con la ResNet50

`make train-resnet compare` (o `kanji-compare runs/cnn runs/resnet50`):

- Comprueba que las dos corridas usan los mismos dibujos, clases, semilla y test.
- **Métricas de test** con intervalo de confianza del 95 % (*bootstrap*, 1000 remuestreos).
- **Test de McNemar exacto**: compara solo los dibujos en que una red acierta y la otra falla. Dos redes con la misma accuracy pueden fallar en imágenes distintas.
- **Coste**: parámetros, tamaño, segundos por época y milisegundos por imagen.
- Figuras: métricas, curvas superpuestas, diferencia de F1 por radical y dibujos en que discrepan.

Salida en `data/runs/compare/`: `comparison.json`, `comparison.md` y figuras.

Comparación: resultados

Precisión en test

Corrida	CNN	ResNet50
Preliminar: 484 dibujos, sin aumento, test $n = 73$	1.000	1.000
Completa: 1612 dibujos, con aumento, test $n = 242$	1.000	*pendiente*

Preliminar: McNemar $p = 1$ (ningún dibujo en que discrepen).

Coste

	CNN	ResNet50
parámetros	1.24 M	23.5 M
tamaño	4.7 MB	89.9 MB
s/época*	5	9
ms/img*	3.8	4.7

* Corrida preliminar, en CPU.

Lectura

Con este split las dos redes aciertan todo: no hay diferencia de precisión que medir, solo de coste. La CNN propia es **19 veces más pequeña**.

Conclusiones

Limitaciones y trabajo futuro

- **El test aleatorio es demasiado fácil**: comparte kanji y escritor con train. Un 100 % no separa a las dos redes.
- **Split por escritor**: dejar a un escritor entero en test (el manifiesto guarda el escritor de cada dibujo). Mediría la generalización a letra nueva.
- **Split por kanji**: dejar kanji enteros fuera de train. Mediría si la red reconoce el radical en kanji que nunca vio.
- **Más escritores**: con 4 personas, la variedad de letra es limitada.
- Con T = 1000, no caben las 4 variantes de cada dibujo: o se sube el máximo, o se usan menos originales.

Reproducibilidad

Comandos

```
make draw           # app de dibujo
make dataset        # manifiestos y aumento
make train          # CNN
make train-resnet   # ResNet50
make compare        # CNN vs ResNet50
make notebooks      # local y Colab
```

- Cada corrida guarda en `data/runs/` `model.pt`, `history.json`, `metrics.json`, `predictions.npz` y sus figuras.
- Notebooks **local** y **Colab** (datos en Drive) con el mismo flujo: mismo split y mismo train aumentado que la línea de comandos.
- El split y el aumento son deterministas con `--seed` (42 por defecto).

Conclusiones

- Construimos el flujo completo: **dibujo** → **etiquetado** → **preprocesado** → **aumento** → **entrenamiento** → **evaluación** → **comparación**.
- El aumento de datos lleva train a **1000 imágenes por radical**, con las clases equilibradas, sin tocar val ni test.
- Nuestra CNN de **1.24 M parámetros** acierta el 100 % del test (242 dibujos).
- Frente a la ResNet50, en la corrida preliminar empatan en precisión, con una red 19 veces más pequeña.
- El siguiente paso es un **test más exigente** (por escritor o por kanji) para medir de verdad la generalización.

¿Preguntas?

土　火　金　木　氵